

Mazes vs Pathfinding (algorithms)

Robert A Lane

Abstract—READ CONC AFTER ABSTRACT TO DOUBLE CHECK IF PAPER IS WORTH USING

This paper demonstrates my experiment to run various pathfinding algorithms against differing maze generation algorithms.

I. INTRODUCTION

This investigation is a 'fun' visual way to compare search algorithms in their speed accuracy when navigating a variety of mazes generated by different algorithms. This should show some of the strengths and weaknesses of both search and maze algorithms. These maze algorithms are essential creating a spanning tree from the grid. As such we can not only use algorithms that generate trees e.g Binary Tree, but also ones that create minimum spanning trees within the graph.

II. RELATED WORK

Research around this topic is focused on maze generation as well as tree search and maze solving solutions.

[11] Vijaya et al discussed the use of mazes within game design. going into how different algorithms can lead to a sense of predictability and therefore boredom when playing. this particular paper explores depth first search, in both solving the maze as well as generating one - both in the form of recursive backtracking algorithms. It concludes that the depth-first search algorithm is an excellent choice for maze generation as it seems to have a brilliant compromise between playability, challenge for the solver with path lengths averaging 122% of the max path length[11] as well as computational efficiency.

[8] Narendran et al explore the effectiveness of various path finding algorithms within a maze. The authors compare fairly basic Breadth-first search, as well as depth-first search with A* search 2 variations of Markov Decision Process as well as an Enhanced wall follower. The paper goes into depth of how many steps taken, memory usage and time complexity of these algorithms. This study is particularly relevant as it is very similar to mine. I am differentiating through comparing the first 3 algorithms and last within mazes generated by different algorithms, as well as these mazes, unlike this work, will not contain loops.

[7] James Buck Goes into varying detail about 15 different algorithms to generate mazes. ranging from the fairly standard recursive backtracker, to the wildly inefficient Aldous-Broder algorithm. This blog and his subsequent book [1] which goes into more detail about how to implement said algorithms and how they work, whilst also discussing how to add braiding, weaves and different grids.

[9] Prim explores what would be known as Prim's algorithm, using this method to generate minimum spanning trees between points. This manuscript references the earlier

paper by Kruskal[6] comparing his solution with Kruskals, and discussing the possibilities against various problems including the traveling salesman problem.

[5] Kozklova et al, compare 3 maze generation algorithms: Prim's, depth first and recursive Division. This summarises that depth first would be the best of the 3 due to the long path ways against Prim's short, but also relatively wiggly passages compared to the recursive division.

[10] Derek Muller, Veritasium, did a video exploring the popular micro mouse competition. Reporting on how the sport developed from the 70s to modern day. The goal is to have your robotic 'mouse' solve a maze as fast as possible. These mice, as well as the competitors that made them, have no prior knowledge of the course so the mice need to be able not only solve the maze quickly but also map out the maze as efficiently as possible. There is an interesting caveat with this competition, because the shortest path will not always be the fastest, which can lead to different problems and solutions than traditional grid-based pathfinders.

[2] Candra et al compare A* and Dijkstra's Algorithm within the context of route mapping between petrol stations. They go into detail of how both algorithms work and compare the outcomes when running them in the same scenarios.

[4] Kim et al, design a development tool to create the 'perfect' perfect maze. They have a quick comparison between Prim's algorithm [9] and recursive backtracker, the former generating many short dead-ends' with the latter, having longer passages with little branching. Kim discusses the flaw with them is that they are completely random as such not hard to design around exact needs for game designers. The solution they found was to create an evaluation function to assign generated mazes a score against user specified values - As such the program can then iterate through generated mazes until a desired maze, or close enough, is made. They expand on the research with their future paper[3]

i think mine is a planning around AI as searching?? AI aren't learning anything

III. IMPLEMENTATION

using (git repos author) repo (reference) we expanded on this to allow for multiple maze algorithms and pathfinders.

IV. THE EXPERIMENT

V. THE MAZE ALGORITHMS

For the algorithms I chose we assume the grid is generated with each cell having all its walls. When a cell is connected, the corresponding wall is removed (each cell is a node, each passage is a connection).

a require "cmp.utils.feedkeys".run(52)

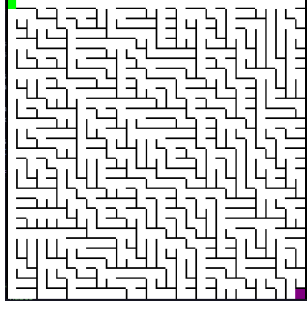


Fig. 1. generation of a binary tree maze given a random seed with a SE bias

Maze algorithms used will be from Jamis Buck's book *Mazes for Programmers*[1] and his blog [7]

section on the maze algorithms/tree/graph gen algorithms I am using will have subsections

A. binaryTree

Algorithm 1 Binary Tree

```

1: procedure BINARYTREE(size) ▷ square grids so size is
   width & height
2:   for  $i \leftarrow 0, size$  do
3:     for  $j \leftarrow 0, size$  do
4:        $Position \leftarrow (i, j)$ 
5:        $Dir \leftarrow AvailableDirs(Position)$ 
6:        $Index \leftarrow Rand(0, len(DirsToMove))$ 
7:        $NextCell \leftarrow DirsToMove[Index]$ 
8:        $Connect(Pos, NextCell)$  ▷ carve passage
   connecting the cells
9:     end for
10:  end for
11: end procedure
12: function AVAILABLEDIRS(pos) ▷ find the next cell to
   carve to
13:   PossibleDirs[]
14:   if pos is within bounds SOUTH then▷ Given SE Bias
15:     AddItem(PossibleDirs,SOUTH)
16:   end if
17:   if pos is within bounds WEST then
18:     AddItem(PossibleDirs,EAST)
19:   end if
20:   return PossibleDirs
21: end function

```

A very simple method of generating a maze is by utilising a binary tree. These mazes, however, are extremely biased. A binary tree maze can only pick 2 directions to move to. This creates a visual effect of a diagonal biased depending on the directions you choose the genral upside to this algorithm is that its firstly incredibly simple to implement1

B. depth first/recursive backtracker

this is a recursive backtracker. The algorithm operates similarly to a depth first search: carving down a branch of

Algorithm 2 Recursive backtracker

```

1: procedure RECURSIVEBACKTRACKER(curCell,size)
2:    $availDirs \leftarrow getAvailDirs(curCell, size)$ ▷ returns
   random ordered list of dirs
3:   for  $dir, availDirs$  do
4:      $NewCell \leftarrow getCell(dir)$ 
5:     if  $!NewCell.Visited$  then
6:        $ConnectCells(NewCell)$ 
7:        $NewCell.Visited \leftarrow TRUE$ 
8:        $RecursiveBacktracker(NewCell, size)$ 
9:     end if
10:  end for
11: end procedure

```

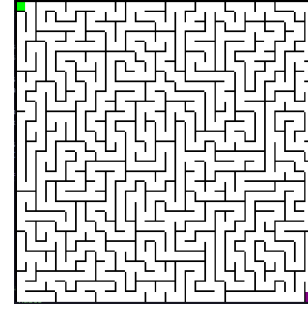


Fig. 2. Generation of recursive backtracker maze given a random seed

a tree all the way till it ends, before looking at the other branches. It then backtracks to the first cell with unvisited neighbours and repeats until there are no unvisited cells. As figure 2 shows, this results in little to no bias in the generation as appose to a binary tree, see figure 1.

VI. THE PATHFINDING ALGORITHMS

A. Brepth First

B. depthFirst

Algorithm 3 depthFirst

```

1: procedure DEPTHFIRST(Path,curCell,size,goal, solved)
2:    $PushBack(Path, curCell)$ 
3:    $availDirs \leftarrow getAvailDirs(curCell, size)$ ▷ returns
   random ordered list of dirs‘
4:   for  $dir, availDirs$  do
5:      $NewCell \leftarrow getCell(dir)$ 
6:     if  $NewCell = goal$  or  $solved = TRUE$  then
7:        $solved \leftarrow TRUE$ 
8:        $return$ 
9:     end if
10:    if  $!NewCell.Visited$  then
11:       $NewCell.Visited \leftarrow TRUE$ 
12:       $RecursiveBacktracker(NewCell, size)$ 
13:    end if
14:  end for
15: end procedure

```

This is a greedy first algorithm which is essentially the same as the Recursive Backtracker :) only difference instead of creating the maze its solving it. This algorithm can be Modified with a heuristic as so the path picking isn't entirely random.

C. A*

D. wall Follower

algorithm from [8]

Algorithm 4

```

1: procedure WALLFOLLOWER(Start, Goal)
2:   pos = Start
3:   dir = SOUTH                                ▷ initial dir
4:   while pos ≠ Goal do
5:     if RightCell is open then
6:       pos = RightCell
7:       dir = rotate(Right)
8:     else if front cell open then
9:       pos = FrontCell
10:    else
11:      rotate(Left)
12:    end if
13:  end while
14: end procedure

```

section on pathfind/tree/graph search algorithms i am using USE ϵ -GREEDY- ζ with binaryTree due to the bias, compare 'with grain vs against grain' will have subsections

VII. RESULTS

VIII. CONCLUSION

IX. FUTURE WORK

REFERENCES

- [1] Jamis Buck. *Mazes for Programmers: Code Your Own Twisty Little Passages*. 1st edition. The Pragmatic Programmers. Dallas, Texas ; The Pragmatic Bookshelf, 2015. ISBN: 978-1-68050-131-5.
- [2] Ade Candra, Mohammad Andri Budiman, and Kevin Hartanto. "Dijkstra's and A-Star in Finding the Shortest Path: A Tutorial". In: *2020 International Conference on Data Science, Artificial Intelligence, and Business Analytics (DATABIA)*. July 2020, pp. 28–32. DOI: 10.1109 / DATABIA50434 . 2020 . 9190342. (Visited on 05/30/2026).
- [3] Paul Hyunjin Kim and Roger Crawfis. "Intelligent Maze Generation Based on Topological Constraints". In: *2018 7th International Congress on Advanced Applied Informatics (IIAI-AAI)*. July 2018, pp. 867–872. DOI: 10.1109/IIAI-AAI.2018.00176. (Visited on 02/16/2026).
- [4] Paul Hyunjin Kim and Roger Crawfis. "The Quest for the Perfect Perfect-Maze". In: *2015 Computer Games: AI, Animation, Mobile, Multimedia, Educational and Serious Games (CGAMES)*. July 2015, pp. 65–72. DOI: 10.1109 / CGames . 2015 . 7272964. (Visited on 02/16/2026).
- [5] Aliona Kozlova, Joseph Alexander Brown, and Elizabeth Reading. "Examination of Representational Expression in Maze Generation Algorithms". In: *2015 IEEE Conference on Computational Intelligence and Games (CIG)*. Aug. 2015, pp. 532–533. DOI: 10.1109/ CIG.2015.7317902. (Visited on 05/30/2026).
- [6] Joseph B Kruskal. "ON THE SHORTEST SPANNING SUBTREE OF A GRAPH AND THE TRAVELING SALESMAN PROBLEM". In: ().
- [7] *Maze Algorithms*. <https://www.jamisbuck.org/mazes/>. (Visited on 02/06/2026).
- [8] Advik Narendran et al. "MazeSolver: Exploring Algorithmic Solutions for Maze Navigation". In: *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. June 2024, pp. 1–8. DOI: 10.1109 / ICCCNT61001 . 2024 . 10725996. (Visited on 05/28/2026).
- [9] R. C. Prim. "Shortest Connection Networks and Some Generalizations". Aug. 1957.
- [10] Veritasium. *The Fastest Maze-Solving Competition On Earth*. May 2023. (Visited on 02/11/2026).
- [11] J. Vijaya et al. "Analysis of Maze Generation Algorithms". In: *2024 5th International Conference on Innovative Trends in Information Technology (ICITIIT)*. Mar. 2024, pp. 1–6. DOI: 10.1109/ICITIIT61487.2024. 10580178. (Visited on 02/16/2026).